



Security Assessment

Final Report



Polymarket V2 Additional changes

May 2026

Prepared for Polymarket

Table of Contents

Project Summary	3
Project Scope	3
Project Overview	3
Protocol Overview	3
Assessment Methodology	5
Findings Summary	6
Severity Matrix	6
Detailed Findings	7
Medium Severity Issues	8
M-01: Users can extract value if a migrated legacy market resolves all NO	8
Informational Severity Issues	9
I-01: Unused MODULE_MASK	9
I-02: PositionIdLib.encode omits arity	10
Disclaimer	12
About Certora	12

Project Summary

Project Scope

Project Name	Repository Link	Commit Hash	Platform
Polymarket V2 Additional changes	https://github.com/Polymarket/polymarket-v2	8009ad9 (initial) ae2548b (fix)	Solidity

Project Overview

This document describes the security review of **Polymarket V2 Additional changes**. The work was undertaken from **May 18th, 2026** to **May 22nd, 2026**.

The following contract list is included in our scope:

```
src/positionManager/PositionManager.sol  
src/modules/BinaryModule.sol  
src/modules/NegRiskModule.sol  
src/modules/CombinatorialModule.sol  
src/exchange/Exchange.sol  
src/utils/AutoRedeemer.sol  
src/routers/Router.sol
```

The team performed a manual audit of all the files in scope. During the manual audit, the Certora team discovered bugs in the code, as listed on the following page.

Protocol Overview

Polymarket V2 is a modular prediction market protocol built on Solidity 0.8.34 with Foundry and Solady. It manages prediction market positions as ERC-1155 tokens through a central **PositionManager** that routes to pluggable market modules (**BinaryModule** and **NegRiskModule**).

Positions use structured IDs that encode all routing information directly in the token ID (`moduleId`, `baseHash`, `conditionIndex`, `outcomeIndex`), eliminating storage lookups for routing. An `OracleAggregator` provides a modular oracle framework with pluggable reporter, disputer, and arbitrator modules. An `Exchange` contract handles EIP-712 signed order matching for position trading. A `BridgeBase` abstract contract enables cross-chain position and collateral transfers.

Core operations (split, merge, redeem) follow an effects-first callback pattern: the module mints output tokens, calls back to the sender to pull input tokens, then verifies the input arrived. The `Router` uses a zero-callback variant where assets are pre-transferred before calling the module. This callback pattern intentionally enables flash-loan-like behavior — callers receive minted tokens before paying for them.

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	-	-	-
High	-	-	-
Medium	1	1	1
Low	-	-	-
Informational	2	2	2
Total	3	3	3

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
Likelihood				

Detailed Findings

ID	Title	Severity	Status
M-01	Users can extract value if a migrated legacy market resolves all NO	Medium	Fixed
I-01	Unused MODULE_MASK	Informational	Fixed
I-02	PositionIdLib.encode omits arity	Informational	Fixed

Medium Severity Issues

M-01: Users can extract value if a migrated legacy market resolves all NO

Severity: Medium	Impact: High	Likelihood: Low
Files: src/modules/NegRiskModule. sol	Status: Fixed	

Description:

If a migrated legacy market resolves all NO, then horizontal merging would give value to the user even though the value of the YES tokens is 0. This is guarded against in the below way:

```
if (_isMigrationEvent(eventId)) {  
    require(resultsSum[eventId] == RESULT_DENOMINATOR,  
MigrationHorizontalMergeRequiresResolution());  
}
```

The issue is that the event ID will only be considered a migration event on the hub chain. On spoke chains, the logic will be skipped. Thus, a user can simply bridge his YES positions to the other chain and redeem them there, bypassing the check.

Recommendations:

Consider refactoring the code to disallow this scenario.

Customer Response:

Fixed in commit [07858ac](#).

Fix Review:

Fix confirmed.

Informational Severity Issues

I-O1: Unused MODULE_MASK

Files:

src/libraries/lds.sol#L33

Description:

To ensure that variables with "dirty" upper bits do not bleed into adjacent packed fields, the protocol correctly applies bitmasks to the parameters before shifting them (e.g., `(_arity & ARITY_MASK) << ARITY_SHIFT`). Previously the protocol used `uint256 constant MODULE_MASK = 0xFF` to do the same thing for the `moduleId`, however, since it's not being used anymore it's completely redundant.

Recommendations:

Remove `MODULE_MASK` constant as it is not used anywhere.

Customer Response:

Fixed in commit [f343168](#).

Fix Review:

Fix confirmed.

I-02: PositionIdLib.encode omits arity

Files:

src/libraries/lds.sol

Description:

`PositionIdLib.encode` fails to include the `arity` field in its parameters and bitwise packing logic:

```
function encode(uint256 _moduleId, uint256 _conditionIndex, uint256
_outcomeIndex, bytes memory _data)
    internal
    pure
    returns (PositionId)
{
    bytes32 baseHash = computeBaseHash(_moduleId, _data);

    return PositionId.wrap(
        (_moduleId << MODULE_SHIFT)
        | ((uint256(baseHash) & BASE_HASH_MASK) << BASE_HASH_SHIFT)
        | ((_conditionIndex & CONDITION_INDEX_MASK) << CONDITION_INDEX_SHIFT)
        | (_outcomeIndex & OUTCOME_MASK)
    );
}
```

Because the `arity` is omitted, the 16 bits starting at `ARITY_SHIFT` (104) are left as zeros.

Currently this function is unused, however, any future use of encoding a `PositionId` for a NegRisk market (which relies on `arity > 0`) will result in an incorrect Id.

Recommendations:

Consider adding an `arity` argument as well.



Customer Response:

Fixed in commit [9376c1e](#).

Fix Review:

Fix confirmed.

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline and is widely used by developers and security researchers during audits and bug bounties.

Certora also provides architectural design reviews, where researchers analyze system design and trust boundaries early to identify structural risks and reduce complexity before implementation.

Certora conducts off-chain audits covering backend services, APIs, frontend, and mobile applications, focusing on vulnerabilities in authentication, data handling, key management, and wallet-related workflows.

Certora also offers Penetration Testing and Red Teaming to simulate real-world attacks and uncover exploitable weaknesses across infrastructure, applications, and business logic.

In addition, Certora provides operational security (OpSec) assessments, reviewing key management, access controls, and operational processes to strengthen overall security posture.

Finally, Certora delivers ongoing monitoring and incident response, enabling continuous threat detection and prevention, alert triage, and coordinated response to active incidents.